

RV Learning Hub (RVLH) LMS — Complete System & Architecture Documentation

Document Version: 1.0.0

Last Updated: September 2026

Target Audience: Engineering Team, Tech Leads, Full-Stack Developers, DevOps Engineers

Production URL: <https://rvlh-lms.vercel.app>

Repository: <https://github.com/VidyaAradhya23/RVLH-LMS.git>

Table of Contents

1. System Overview & Architecture
2. Technology Stack & Modules
3. Project Directory & File Structure
4. Authentication, Security & RBAC
5. Role-Based Feature Catalog
 - 5.1 Student Portal
 - 5.2 Faculty Portal
 - 5.3 Admin Portal
6. Data Models & MongoDB Schemas
7. State Management & Data Synchronization (`/api/sync`)
8. Complete REST API Reference
9. Local Development & Environment Setup
10. Production Deployment Guide (Vercel & MongoDB Atlas)
11. Developer Guide for Future Extensions

1. System Overview & Architecture

RV Learning Hub (RVLH) LMS is a full-featured, cloud-native Learning Management System designed for coaching institutions and colleges offering competitive exam preparations (such as **JEE Advanced, JEE Main, NEET UG, KCET, and Commerce Programmes**).



Key Architectural Principles

- 1. Ultra-Fast Single Page Application (SPA):** The frontend is written in optimized, dependency-free Vanilla JavaScript with modular page components registered under a global `PAGES` dictionary, compiled and bundled via **Vite**.
- 2. Unified Batch Synchronization (`/api/sync`):** Instead of issuing 15+ disparate HTTP requests on page load, the frontend makes a single authenticated batch call to `/api/sync`, which uses MongoDB's concurrent `Promise.all()` to hydrate all course, video, test, doubt, material, and notification states in one round-trip.
- 3. Optimistic UI with Real-time Broadcasting:** Actions (like approving a video, creating a test, or asking a doubt) update the local UI immediately and broadcast real-time events across roles.
- 4. Serverless & Monorepo Deployability:** Built to run either as standard independent Node + Vite services locally or as a unified Serverless app on **Vercel** with `@vercel/node` and `@vercel/static-build`.

2. Technology Stack & Modules

Frontend Stack (/web-app)

Technology / Module	Version	Purpose
Vanilla JavaScript (ES6+)	Native	Core business logic, dynamic DOM generation, client routing, modal handlers
HTML5 & Modern CSS3	Native	Semantic layout, CSS variables/tokens, glassmorphism, responsive Grid & Flexbox
Vite	^6.3.5	Next-generation frontend build tool, ultra-fast HMR dev server, asset bundler
Google Fonts	CDN	Typography: Syne (display & brand headers) and Inter / DM Sans (body text)

Backend Stack (/backend)

Package / Dependency	Version	Purpose
Node.js	>=18.x	JavaScript runtime environment
Express.js	^5.2.1	RESTful API web framework
Mongoose	^9.9.4	MongoDB Object Data Modeling (ODM) library with schema validation & indexing
jsonwebtoken (JWT)	^9.0.3	Secure stateless user session management & token authentication
bcryptjs	^3.0.3	One-way salt hashing for user password storage
cors	^2.8.6	Cross-Origin Resource Sharing middleware
dotenv	^17.4.2	Environment variables loader (.env)
nodemon	^3.1.14	Development auto-restart on file modifications

Cloud & Infrastructure

- **Hosting Platform:** Vercel (Frontend static edge distribution + Backend Serverless Functions).
 - **Database:** MongoDB Atlas (AWS cluster with automated failover and M0/M10 pooling).
 - **Version Control:** Git & GitHub (main branch automated CI/CD via Vercel GitHub App).
-

3. Project Directory & File Structure

```
d:\Final LMS\
├── backend/
│   ├── .env                # Environment configuration (PORT, MONGO_URI, JWT_SECRET)
│   ├── index.js            # Master Express backend (Mongoose models, Auth, APIs, Seed)
│   ├── package.json        # Backend dependencies and startup scripts
│   └── package-lock.json
├── web-app/
│   ├── index.html          # Main SPA entry point (Navigation, Modals, Auth Forms)
│   ├── main.js             # Core frontend script (Router, State, Page Builders, APIs)
│   ├── style.css           # Comprehensive design system (Tokens, Components, Dark Theme)
│   ├── vite.config.js      # Vite configuration with proxy rules (/api -> :5000)
│   ├── package.json        # Frontend dependencies & build commands
│   ├── package-lock.json
│   └── dist/               # Production build output generated by `npm run build`
├── vercel.json             # Monorepo build and serverless routing for Vercel
├── rv-lms-enhanced.html    # Standalone single-file prototype backup
├── LMS_SYSTEM_DOCUMENTATION.md # Master system & technical documentation (This file)
└── .gitignore              # Git ignored paths (node_modules, .env, dist)
```

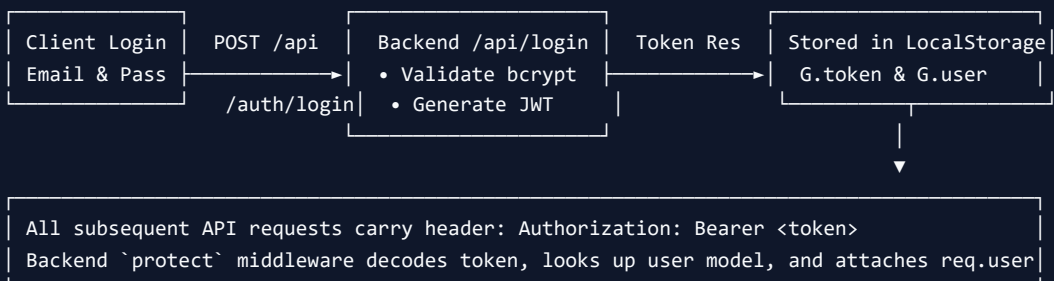
4. Authentication, Security & RBAC

1. Role-Based Access Control (RBAC)

The LMS natively enforces three distinct roles:

- student** — Enrolled learner with read access to assigned batch materials, test taking, doubt posting, and live class participation.
- faculty** — Instructor with rights to upload lecture videos, schedule live sessions, create & grade tests, resolve doubts, and upload notes (subject to admin approval).
- admin** — Superuser with complete institutional oversight: student/faculty CRUD, course creation, approvals pipeline, notifications broadcast, and platform reports.

2. Authentication Flow



3. Demo Credentials Seeded in Database

Role	Email	Password	Default Campus / Batch
Student	arjun@rvhub.com	Student@123	RV Jayanagar (JEE Advanced)
Faculty	priya@rvhub.com	Faculty@123	RV Jayanagar (Physics)
Admin	admin@rvhub.com	Admin@123	Institution Wide

5. Role-Based Feature Catalog

5.1 Student Portal

Feature	Description	File / Handler
Dashboard	Displays active courses, attendance rate, study streak, upcoming live lecture timer, continue-watching list, and recent notifications.	PAGES['student_dashboard'] in main.js
My Courses	Interactive course directory showing module progress (%), lecture counts, PDF notes, and subject filters (Physics, Chemistry, Maths, Biology).	PAGES['student_courses'] in main.js
Video Lectures	Full-featured video lecture player supporting chapter timestamps, playback speed (0.75x–2x), attached DPP worksheets, and lecture notes.	PAGES['student_videos'], openVideoPlayerModal
Live Classes	Weekly live lecture calendar with live join links (Zoom/Google Meet integration), faculty info, and past class recordings archive.	PAGES['student_live'], joinLiveClass
AI Doubt Solver	Dual-mode doubt interface: instant AI-assisted explanation or direct post to faculty with photo attachment and solution tracking.	PAGES['student_doubts'], askDoubtModal
Tests & Practice DPPs	Online test engine with timed countdowns, interactive question palette (Answered, Flagged, Unvisited), auto-evaluation, scorecards, and solution explanations.	PAGES['student_tests'], startTestEngine
Study Materials	Downloadable PDF study repository indexed by subject, chapter, and previous years question papers (PYQs).	PAGES['student_materials'], downloadMaterialFile
Leaderboard	Batch-wide gamified leaderboard displaying ranks, average test scores, attendance %, and study streaks.	PAGES['student_leaderboard']
Profile & Settings	Personal info, registered batch, campus affiliation, login security, and notification preferences.	PAGES['student_profile']

5.2 Faculty Portal

Feature	Description	File / Handler
Faculty Dashboard	Overview of assigned batches, total students taught, pending doubt count, and upcoming scheduled live classes.	<code>PAGES['faculty_dashboard']</code>
Lecture Uploads	Interface for uploading recorded video lectures, tagging subjects, assigning batches, and submitting for Admin approval.	<code>PAGES['faculty_videos']</code> , <code>openFacultyVideoUploadModal</code>
Live Class Scheduler	Scheduler for hosting new live classes, generating meeting URLs, selecting target batches, and notifying students.	<code>PAGES['faculty_live']</code> , <code>openScheduleLiveModal</code>
Test & DPP Creator	Test authoring tool allowing faculty to define question counts, time limits, positive/negative marking rules, and publish dates.	<code>PAGES['faculty_tests']</code> , <code>openCreateTestModal</code>
Doubt Resolution	Faculty inbox to review student doubt tickets, write detailed step-by-step explanations, or attach formula guides.	<code>PAGES['faculty_doubts']</code> , <code>openResolveDoubtModal</code>
Material Publisher	Upload PDFs, formula cheat sheets, and assignment DPPs for assigned courses.	<code>PAGES['faculty_materials']</code> , <code>openUploadMaterialModal</code>

5.3 Admin Portal

Feature	Description	File / Handler
Executive Dashboard	High-level metrics: Total Students (1,200+), Active Faculty (40+), Total Tests Conducted, Course Enrollment Visualizer, and Real-time Activity Stream.	<code>PAGES['admin_dashboard']</code>
User Management	Complete directory of Students and Faculty. Features: Add Student/Faculty, Edit Profile, Deactivate/Activate account, filter by Campus/Course, and CSV export.	<code>PAGES['admin_users']</code> , <code>openAddStudentModal</code> , <code>openStudentEditModal</code>
Batch Management	Roster view for each batch (e.g. JEE Advanced A, NEET Morning), capacity tracker, assigned faculty, and bulk student exports.	<code>openBatchManageModal</code> , <code>batchExport</code>
Course Manager	Create, edit, and publish new academic programs with subject definitions, durations, and faculty leads.	<code>PAGES['admin_courses']</code> , <code>openCreateCourseModal</code>
Approvals Center	Review workflow for videos, study materials, and tests submitted by faculty. Admin can approve or reject with feedback before public display.	<code>PAGES['admin_approvals']</code> , <code>reviewApprovalItem</code>
Announcements Center	Broadcast critical announcements institution-wide, to students only, or to faculty only, with urgency badges.	<code>PAGES['admin_announcements']</code> , <code>openCreateAnnouncementModal</code>
System Reports	Generates exportable analytics on student attendance, test performance averages, and batch progress.	<code>openGenerateReportModal</code>

6. Data Models & MongoDB Schemas

All data models are defined in `backend/index.js` using Mongoose:

```

// 1. Student Schema
StudentSchema = {
  name: String,
  email: { type: String, unique: true, required: true },
  phone: String,
  password: { type: String, required: true }, // bcrypt hashed
  role: { type: String, default: 'student' },
  batch: String, // e.g., 'JEE Advanced'
  roll: { type: String, unique: true }, // e.g., 'RV2024001'
  streak: { type: Number, default: 1 },
  avgScore: { type: Number, default: 0 },
  campus: String, // e.g., 'RV Jayanagar'
  gender: String,
  dob: String,
  st: { type: String, default: 'active' } // 'active' | 'warning' | 'deactivated'
}

// 2. Teacher (Faculty) Schema
TeacherSchema = {
  name: String,
  email: { type: String, unique: true, required: true },
  phone: String,
  password: { type: String, required: true },
  role: { type: String, default: 'faculty' },
  subject: String, // 'Physics' | 'Chemistry' | 'Mathematics' | 'Biology'
  emp: { type: String, unique: true }, // e.g., 'RVF001'
  campus: String,
  batch: String,
  rat: { type: String, default: '4.8' },
  st: { type: String, default: 'active' }
}

// 3. Admin Schema
AdminSchema = {
  name: String,
  email: { type: String, unique: true, required: true },
  phone: String,
  password: { type: String, required: true },
  role: { type: String, default: 'admin' },
  dept: { type: String, default: 'Academic Operations' },
  st: { type: String, default: 'active' }
}

// 4. Course Schema
CourseSchema = {
  e: String, // Emoji icon e.g., '🧠'
  title: { type: String, required: true },
  desc: String,
  videos: { type: Number, default: 0 },
  materials: { type: Number, default: 0 },
  quizzes: { type: Number, default: 0 },
  col: String, // Gradient e.g., 'linear-gradient(90deg,#6c47ff,#a855f7)'
  p: { type: Number, default: 0 }, // Progress percentage
  done: { type: Number, default: 0 },
  total: { type: Number, default: 60 },
  maxSt: { type: Number, default: 150 },
  fac: String, // Lead faculty name
  cat: String, // 'JEE' | 'NEET' | 'Commerce' | 'Foundation'
  dur: String, // '1 Year' | '2 Years' | 'Crash Course'
  subjects: [String],
  rating: { type: Number, default: 4.8 },
  reviews: { type: Number, default: 0 },

```



```

    pub: { type: Boolean, default: true }
  }

// 5. Video Lecture Schema
VideoSchema = {
  thumb: String, // Emoji or image thumbnail
  title: { type: String, required: true },
  dur: String, // Duration e.g., '48 min'
  views: { type: Number, default: 0 },
  date: String,
  fac: String,
  sub: String,
  course: String,
  url: String, // Stream or YouTube embed URL
  notes: String
}

// 6. Test / Quiz Schema
TestSchema = {
  n: { type: String, required: true }, // Test Title
  type: { type: String, default: 'DPP' }, // 'Mock' | 'DPP' | 'Chapter Test'
  subject: String,
  qs: { type: Number, default: 20 }, // Number of questions
  duration: { type: String, default: '60 min' },
  marksCorrect: { type: String, default: '+4' },
  marksWrong: { type: String, default: '-1' },
  batch: String,
  startDate: String,
  endDate: String,
  deadline: String,
  att: { type: Number, default: 0 }, // Total students attempted
  pub: { type: Boolean, default: true },
  fac: String
}

// 7. Doubt Schema
DoubtSchema = {
  student: String,
  roll: String,
  sub: String,
  q: { type: String, required: true }, // Question text
  ans: String, // Faculty answer
  st: { type: String, default: 'open' }, // 'open' | 'resolved'
  ansBy: String, // Faculty responder name
  ansDate: String
}

// 8. Approvals Workflow Schema
ApprovalSchema = {
  type: { type: String, enum: ['video', 'material', 'test'] },
  title: String,
  faculty: String,
  course: String,
  subject: String,
  batch: String,
  st: { type: String, enum: ['pending', 'approved', 'rejected'], default: 'pending' },
  reason: String
}

// 9. Announcement Schema
AnnouncementSchema = {
  title: String,
  body: String,

```

```

cat: String, // 'Exam' | 'Event' | 'General' | 'Academic'
date: String,
urgent: { type: Boolean, default: false },
target: { type: String, default: 'all' }, // 'all' | 'students' | 'faculty'
draft: { type: Boolean, default: false }
}

```

7. State Management & Data Synchronization

The `/api/sync` Engine

To ensure high responsiveness and zero unnecessary loading states, the frontend uses an event-driven synchronization model.

```

// web-app/main.js
async function syncLMSData() {
  try {
    const data = await api('/api/sync');
    if (!data) return;

    // Hydrate Global Window State
    window.LMS_COURSES      = data.courses      || [];
    window.LMS_VIDEOS       = data.videos       || [];
    window.LMS_LIVE         = data.liveClasses  || [];
    window.LMS_DOUBTS       = data.doubts       || [];
    window.LMS_MATERIALS    = data.materials    || [];
    window.LMS_ANNOUNCEMENTS = data.announcements || [];
    window.LMS_TESTS        = data.tests        || [];
    window.LMS_APPROVALS    = data.approvals    || [];
    window.ADMIN_STUDENTS   = data.students     || [];
    window.ADMIN_FACULTY    = data.teachers     || [];
    window.LMS_NOTIFICATIONS = data.notifications || [];

    updateHeaderNotificationBadge();
  } catch (err) {
    console.warn('Sync warning:', err);
  }
}

```

Real-Time Event Broadcast

The backend defines `broadcastRealtimeEvent(eventType, payload)` which triggers instant state invalidation on client subscribers:

- `COURSE_CREATED` / `COURSE_UPDATED`
- `VIDEO_UPLOADED` / `APPROVAL_STATUS_CHANGED`
- `TEST_CREATED` / `DOUBT_RESOLVED`
- `ANNOUNCEMENT_POSTED` / `NOTIFICATION_SENT`

8. Complete REST API Reference

All protected endpoints require the HTTP header:

```
Authorization: Bearer <JWT_TOKEN>
```

Authentication Routes

Method	Endpoint	Access	Description
POST	/api/auth/login	Public	Authenticates student, faculty, or admin; returns JWT and user profile.
POST	/api/auth/register	Admin / Internal	Creates a new user record with hashed password and initial role metadata.
GET	/api/auth/me	Protected	Returns the authenticated user's current session profile.
PUT	/api/auth/users/:id	Admin	Updates user information (batch, campus, email, roll number, phone).
PUT	/api/auth/users/:id/status	Admin	Toggles user status (active <-> warning / deactivated).
DELETE	/api/auth/users/:id	Admin	Permanently deletes a user from the database.

Core Data & Sync

Method	Endpoint	Access	Description
GET	/api/sync	Protected	Master Batch Sync: returns all collections filtered for the caller's role in a single call.

Courses & Academics

Method	Endpoint	Access	Description
GET	/api/courses	Protected	Retrieves list of all active courses.
POST	/api/courses	Admin	Creates a new course program with curriculum and subjects.
PUT	/api/courses/:id	Admin	Edits an existing course's metadata and description.
POST	/api/courses/:id/enroll	Student	Enrolls the authenticated student in a course.

Video Lectures & Live Classes

Method	Endpoint	Access	Description
GET	/api/videos	Protected	Fetches video lecture catalog for user's enrolled batch.
POST	/api/videos	Faculty / Admin	Uploads/registers a video lecture (creates approval request if faculty).
GET	/api/live	Protected	Returns scheduled and ongoing live lecture streams.
POST	/api/live	Faculty / Admin	Schedules a new live class and generates broadcast notifications.

Doubts & Tests

Method	Endpoint	Access	Description
GET	/api/doubts	Protected	Fetches doubt threads (filtered by student or all for faculty/admin).
POST	/api/doubts	Student	Submits a new doubt inquiry with subject tagging.
PUT	/api/doubts/:id/resolve	Faculty / Admin	Resolves doubt ticket with detailed answer.
GET	/api/tests	Protected	Lists scheduled tests and practice DPP worksheets.
POST	/api/tests	Faculty / Admin	Creates a new test paper with timer and scoring rules.
POST	/api/tests/:id/submit	Student	Submits completed test answers for evaluation.

Approvals, Announcements & Notifications

Method	Endpoint	Access	Description
GET	/api/approvals	Admin	Lists pending faculty content uploads awaiting review.
PUT	/api/approvals/:id	Admin	Approves or rejects an uploaded video, test, or study material.
GET	/api/announcements	Protected	Fetches targeted announcements for user role.
POST	/api/announcements	Admin	Publishes a new institutional announcement.
GET	/api/notifications	Protected	Fetches user notifications.
PUT	/api/notifications/:id/read	Protected	Marks notification as read.

9. Local Development & Environment Setup

Prerequisites

- **Node.js**: v18.0.0 or higher ([Download Node.js](#))
- **npm**: v9.0.0 or higher
- **MongoDB Atlas** account or local MongoDB Community Server running on `mongodb://127.0.0.1:27017`

Step 1: Clone the Repository

```
git clone https://github.com/VidyaAradhya23/RVLH-LMS.git
cd "RVLH-LMS"
```

Step 2: Configure Environment Variables

Inside the `backend/` directory, create or verify `.env`:

```
PORT=5000
JWT_SECRET=your_jwt_secret_key_here_at_least_32_chars
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/rvlms?retryWrites=true&w=majority
```

Step 3: Install Dependencies

Open two terminal windows:

Terminal 1 (Backend Server):

```
cd backend
npm install
npm run dev
# Server starts on http://localhost:5000
```

Terminal 2 (Frontend Client):

```
cd web-app
npm install
npm run dev
# Vite dev server starts on http://localhost:3000
```

Step 4: Access the Application

Open your browser and navigate to:

```
http://localhost:3000
```

Use the role buttons at the top of the login card (`Admin` , `Faculty` , `Student`) to automatically test any persona.

10. Production Deployment Guide

The project is structured with a root `vercel.json` for deployment to **Vercel**.

vercel.json Configuration

```
{
  "version": 2,
  "builds": [
    {
      "src": "web-app/package.json",
      "use": "@vercel/static-build",
      "config": {
        "distDir": "dist"
      }
    },
    {
      "src": "backend/index.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    { "src": "/api/(.*)", "dest": "/backend/index.js" },
    { "src": "/assets/(.*)", "dest": "/web-app/assets/$1" },
    { "src": "/(.*)", "dest": "/web-app/$1" }
  ]
}
```

Deploying to Vercel

1. Link your GitHub repository (`VidyaAradhya23/RVLH-LMS`) to Vercel.
2. In the **Vercel Dashboard > Project Settings > Environment Variables**, add:
 - `MONGO_URI` = `mongodb+srv://<username>:<password>@.../rvlms`
 - `JWT_SECRET` = `your_production_secret`
3. Push to `main` branch:

```
git add -A
git commit -m "deploy update"
git push origin main
```

Vercel will trigger a production build and deploy to `https://rvlh-lms.vercel.app`.

11. Developer Guide for Future Extensions

How to Add a New Page to the LMS

To add a new page (e.g., `student_attendance` or `faculty_assignments`):

1. **Register the Navigation Item in `web-app/main.js`**: Add your entry into `NAV[role]`:

```
NAV.student.push({ id: 'attendance', label: 'Attendance', icon: '📅' });
```

2. **Add the Page Title in `PAGE_TITLES`**:

```
PAGE_TITLES['attendance'] = { title: 'My Attendance', sub: 'Track your subject-wise lecture attendance' };
```

3. Define the Page Renderer in `PAGES` dictionary:

```
PAGES['student_attendance'] = function() {
  var attendanceList = window.LMS_ATTENDANCE || [];
  // Return HTML template string
  return '<div class="card"><div class="card-title">Attendance Records</div>'
    + attendanceList.map(a => '<div class="list-item">${a.date} - ${a.sub} - ${a.status}</div>').join('')
    + '</div>';
};
```

4. Add Backend Schema & API in `backend/index.js` (If New Collection Needed):

```
const AttendanceSchema = new mongoose.Schema({
  student: String, roll: String, date: String, sub: String, status: String
}, { timestamps: true });
const Attendance = mongoose.model('Attendance', AttendanceSchema);

app.get('/api/attendance', protect, async (req, res) => {
  res.json(await Attendance.find({ student: req.user.name }).sort({ createdAt: -1 }));
});
```

5. Include the Collection in `/api/sync`: Add `Attendance.find().lean()` to the `Promise.all` block in `app.get('/api/sync')` so the frontend automatically gets fresh data upon reload.

Code Best Practices

- **No Direct DOM Polling:** Always use `syncLMSData()` and `loadPage(G.page)` instead of manually manipulating disparate DOM nodes.
 - **Modals:** Use `openDetail(title, bodyHtml, footerHtml)` for all standard modal dialogues and `closeModal('modal-detail')` for dismissals.
 - **Toasts:** Use `toast('Message here', '✅' | '⚠️' | '❌')` for consistent user feedback.
 - **Security:** Never expose plain-text passwords or JWT secrets in client-side code. Always pass JWT in the `Authorization` header.
-

End of Documentation. For questions or architecture reviews, contact the RVLH Platform Engineering Team.